

L05: Support Vector Machines

CSci 574 Machine Learning (Géron) Ch. 5

Derek Harter

Department of Computer Science
East Texas A&M University

Fall 2025

Large Margin Classification

When two classes can be separated by a line (hyperplane).

- They are **linearly separable**

SVM classifier fits the widest possible street.

- This is called **large margin classification**

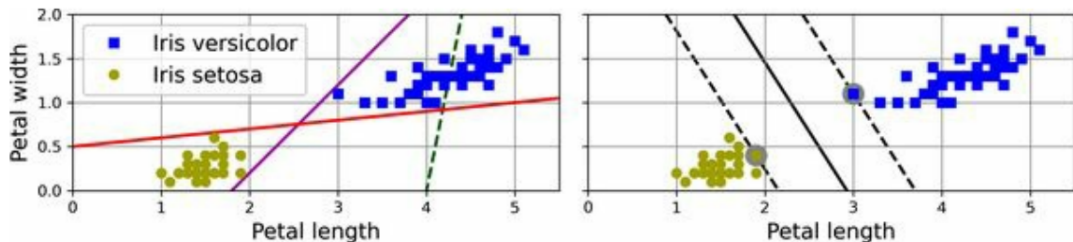


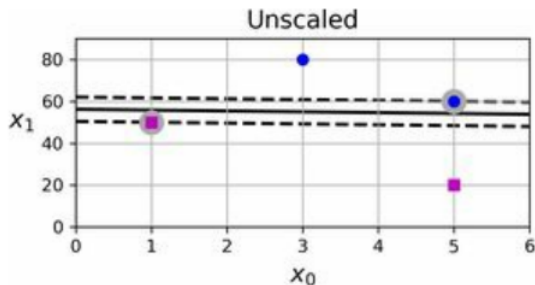
Figure 1: Large margin classification (Géron, 2023, pg.287)

Linear Support Vector Machine

- Notice that adding more training instances “off the street” will not affect the decision boundary at all.
 - It is fully determined (or “supported”) by the instances located on the edge of the street
 - These instances are called the **support vectors**
- A Support Vector Machine (SVM) is a machine learning model capable of performing linear or nonlinear classification and regression.
- It works by identifying support vectors to determine the largest margin decision boundary

SVMs are Sensitive to Scaling

In the left plot vertical scale is much larger than the horizontal scale, gives a almost horizontal large margin with these support vectors.



In the right plot after standard feature scaling, the decision boundary had a much different slope.

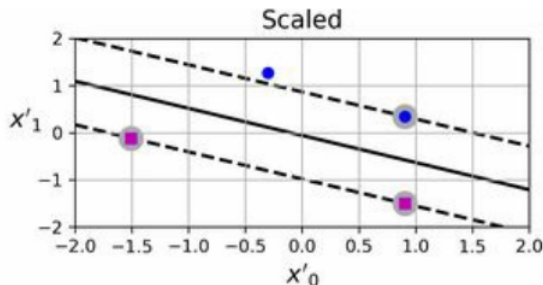


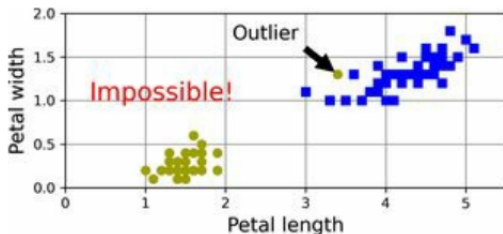
Figure 2: SVM sensitivity to feature scales. (Géron, 2023, pg.288)

Soft Margin Classification

If we strictly require all instances off street and on correct side of margin, this is called **hard margin classification**.

Only possible when data are linearly separably.

- Figure 3 left, impossible to find hard margin, not linearly separable.



Also sensitive to outliers.

- Figure 3 right, decision boundary ends up very different if there is an outlier here, might not generalize.

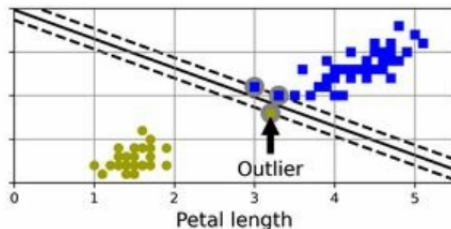


Figure 3: Hard margin sensitivity to outliers. (Géron, 2023, pg.288)

Soft Margin Classification

- To avoid both, find balance between keeping street as large as possible and limiting the **margin violations**
 - This is **soft margin classification**
- SVM use regularization hyperparameter C
 - When C is low, large margin, more margin violations allowed, but if too low can cause underfitting.
 - When C is high, small margins, less margin violations allowed, but can overfit and be sensitive to outliers

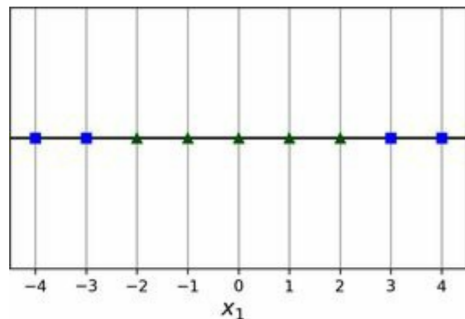
SVM C Regularization

If your SVM model is overfitting, you can regularize it by reducing C , leading to softer margins which will tend to be better at generalizing performance to unseen data.

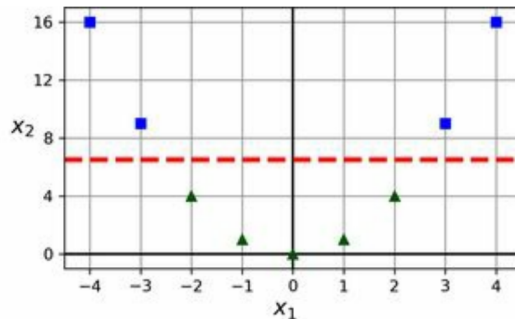


Nonlinear SVM Classification

- Many datasets are not even close to being linearly separable.
- One approach is to add more polynomial feature combinations of the original features.
- This can result in transforming data into a linearly separable dataset.



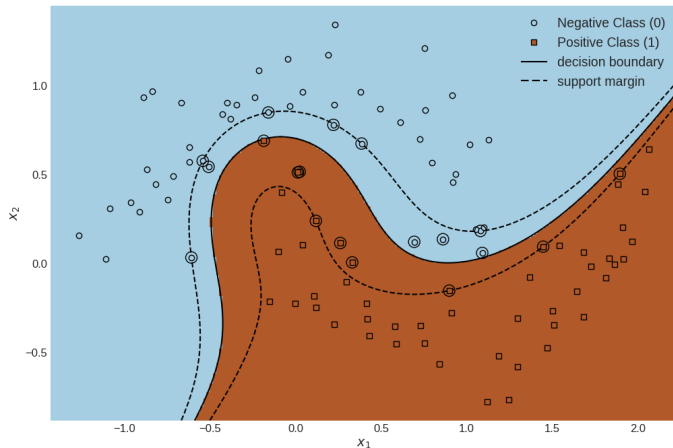
Single feature x_1 is not linearly separable.



But if we add new polynomial feature $x_2 = (x_1)^2$, resulting 2D dataset is linearly separable.

Adding Nonlinear Features by Hand

As we have done previously for logistic regression, add in nonlinear polynomial features by hand.



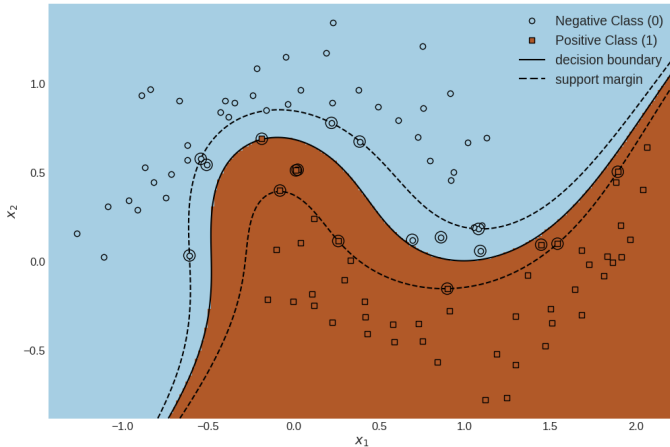
```
# adding degree 3 polynomials combinations  
# to 2 feature nonlinear dataset  
# makes resulting nonlinear features  
# linearly separable  
X, y = make_moons(n_samples=100, noise=0.15)  
  
polynomial_svm_clf = Pipeline([  
    ("poly_features", PolynomialFeatures(degree=3)),  
    ("scaler", StandardScaler()),  
    ("clf", SVC(kernel='linear', C=10, max_iter=10000))  
)  
  
polynomial_svm_clf.fit(X, y)
```

Figure 4: Linear SVM classifier using polynomial features.

Polynomial Kernel, the **Kernel Trick**

- Adding polynomial features can work with all sorts of ML (not just SVMs)
 - But at low polynomial degree, cannot deal with complex datasets
 - And at high polynomial degree, creates huge (combinatorial explosion) number of features (slow).
- When using SVM, can use the **kernel trick**
 - Can give same result as if added many polynomial features, without the computational costs of actually adding them.
- The kernel trick is built-in to the `scikit-learn` SVC class.

SVC Polynomial Kernel



```
# using a degree 3 polynomial kernel for a SVC,  
# this implementation will be much faster for large  
# polynomial degrees / large datasets  
from sklearn.svm import SVC
```

```
poly_kernel_svm_clf = Pipeline([  
    ("scaler", StandardScaler()),  
    ("clf", SVC(kernel="poly", degree=3, coef0=1, C=5))  
])
```

```
poly_kernel_svm_clf.fit(X, y)
```

Figure 5: SVC Using degree 3 polynomial kernel (implemented using the kernel trick).

SVM Kernel Regularization

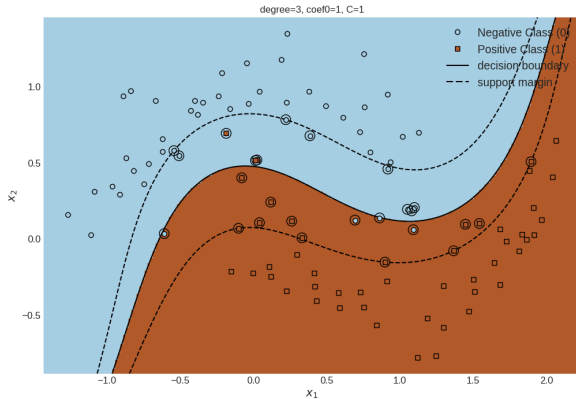


Figure 6: SVC Using degree 3 polynomial kernel, large regularization, if need to underfit.

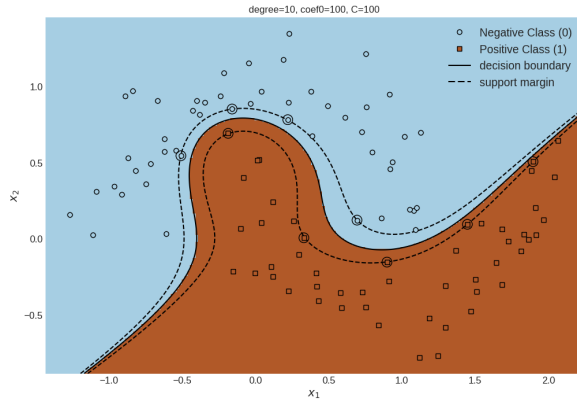


Figure 7: SVC Using degree 10 polynomial kernel, small regularization, if need to overfit.

Géron, A. (2023). *Hands-on machine learning with scikit-learn, keras and tensorflow* (third).
O'Reilly Media, Inc.